

KR 유튜브 트렌드 대시보드

단계별 개발 체크리스트 PRD

Vibe Coding Edition — 인기 영상 · 구독자 · 증가율 · 연령대 분석 대시보드

항목	내용
버전	v1.0 (UI 샘플 확정 기반)
작성일	2026년 5월
개발 방식	바이브 코딩 (Claude Code / Cursor)
핵심 데이터	YouTube Data API v3 (무료, 일 10,000유닛)
개발 기간	5주 (Phase 1: 2주 / Phase 2: 3주)
배포 환경	정적 HTML + FastAPI 백엔드 (Oracle Cloud Free)
월 예상 비용	\$0 (YouTube API 무료 티어 내 운영)

1. 프로젝트 개요

1.1 배경 및 목적

한국 유튜브의 최신 동향을 한눈에 파악할 수 있는 대시보드를 구축한다. 인기 영상 순위(카테고리별 TOP 10), 채널 구독자 순위, 일간·주간·월간 구독자 증가율, 연령대별 콘텐츠 선호 분석을 제공하며, 1위 영상은 대시보드 내 인라인 재생을 지원한다.

1.2 핵심 목표

목표	설명	달성 기준
카테고리별 TOP 10	전체/음악/게임/엔터/뉴스/교육/뷰티/먹방/여행/IT 각 10건	카테고리 전환 시 독립 데이터 표시
구독자 순위	한국 주요 채널 구독자 수 TOP 10	channels.list 배치 1유닛으로 처리
증가율 분석	일간/주간/월간 구독자 증가 TOP 10	DB 스냅샷 차이 계산으로 구현
연령대 분석	10대~50대 선호 콘텐츠 & 추천 영상	KISDI 연구 매핑 테이블 적용
인라인 재생	1위 영상 → 페이지 내 유튜브 embed	2~10위는 새 탭 유튜브 이동
쿼터 최소화	일 사용량 10,000유닛 10% 이내 유지	캐싱·배치 전략 적용

1.3 확정된 UI 레이아웃 (샘플 기준)

헤더바: [로고] [갱신시각] [v 최신뉴스 갱신]	
탭 메뉴: 인기 영상 순위 / 구독자 순위 / 증가율 / 연령대	
필터: [일간 주간 월간] [전체 음악 게임 엔터 뉴스 교육 뷰티 먹방 여행 IT]	
1위 인라인 플레이어 (embed)	2~10위 썸네일 리스트 (클릭 → 새탭 유튜브) 번호 / 썸네일 / 제목 / 조회수

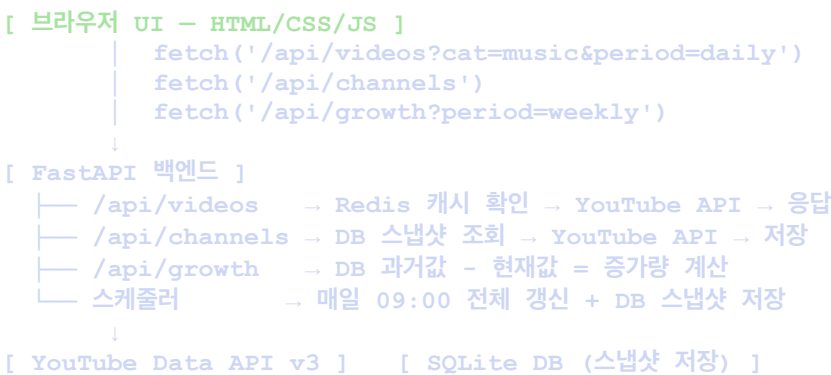
2. 데이터 아키텍처 & API 설계

2.1 YouTube Data API v3 쿼타 전략

⚠ YouTube API는 하루 기본 10,000유닛이 제공됩니다. search.list는 100유닛/회로 매우 비싸므로 절대 사용하지 않습니다.

API 메서드	유닛 비용	용도	호출 전략
videos.list (chart=mostPopular)	1유닛/회	카테고리별 인기 영상 수집	카테고리×3기간 = 30회 → 30유닛/일 캐시1h
channels.list (id 배치 50개)	1유닛/회	채널 구독자 수 수집	50개 배치 1회 → 1유닛/일 캐시24h
search.list	100유닛/회	❌ 사용 금지	video ID 사전 확보 후 videos.list 사용
합계 예상	~170유닛/일	전체 데이터 갱신	10,000유닛의 1.7% 사용

2.2 전체 시스템 아키텍처



2.3 구독자 증가율 계산 방식

YouTube API는 현재 구독자 수만 제공합니다. 증가율은 아래 방식으로 계산합니다.

```
# 매일 09:00 스케줄러 실행
오늘 구독자 수 = channels.list API 호출
DB에 {channel_id, subscriber_count, timestamp} 저장

# 증가량 계산
일간 증가 = 오늘 저장값 - 어제 저장값
주간 증가 = 오늘 저장값 - 7일 전 저장값
월간 증가 = 오늘 저장값 - 30일 전 저장값
증가율(%) = 증가량 / 기준일 구독자 수 × 100
```

2.4 연령대 매핑 테이블 (KISDI 연구 기반)

💡 YouTube API는 타 채널의 연령대 데이터를 제공하지 않습니다. KISDI·갤럽·나스미디어 연구 데이터를 기반으로 카테고리-연령대 매핑 테이블을 구현합니다.

YouTube 카테고리	주 시청 연령대	근거 데이터	API categoryId
게임 (Gaming)	10~20대	KISDI 20대 1위 장르	ID: 20
음악 (Music)	10~20대	전 연령 공통, 10~20대 집중	ID: 10
뷰티/패션 (Howto)	10~30대 여성	나스미디어 2025 NPR	ID: 26
엔터테인먼트	20~40대	KISDI 전 연령 1~2위	ID: 24
교육 (Education)	20~30대	자기계발 수요층	ID: 27
먹방 (Food)	20~40대	전 연령 선호, 30대 집중	ID: 26
여행 (Travel)	30~50대	여행 지출 주도층	ID: 19
뉴스/정치 (News)	40~60대	KISDI 50대 1위 장르	ID: 25
IT/과학 (Science)	20~40대	디지털 소비 주도층	ID: 28

3. UI 상세 설계

3.1 4개 탭 구성

탭	주요 기능	핵심 인터랙션
인기 영상 순위	카테고리 × 기간 필터 / 1위 인라인 재생 / 2~10위 새탭	카테고리 탭 클릭 → 독립 데이터 10건 로드
구독자 순위	한국 채널 TOP 10 / 구독자수·조회수 정렬	정렬 전환 버튼
구독자 증가율	일간·주간·월간 증가 TOP 10 / 스파크라인 추세	기간 전환 버튼 → 증가량·증가율 재계산
연령대 분석	10대~50대 시청 비중 / 연령별 선호 콘텐츠 / 추천 영상 5건	연령대 필터 클릭 → 선호도·추천 영상 갱신

3.2 영상 재생 UX 설계

순위	재생 방식	구현 방법	이유
1위	페이지 내 인라인 재생	YouTube embed iframe (autoplay=1)	가장 인기 영상 → 즉시 시청 유도
2~10위	새 탭으로 유튜브 이동	<a href target=_blank>	순위 목록 유지하면서 다른 영상 탐색
연령대 탭 추천 영상	썸네일 클릭 → 새 탭	<a href target=_blank>	분석 화면 유지

3.3 카테고리 필터 목록

카테고리 라벨	YouTube categoryId	색상 배지	주 연령대
전체	mostPopular (regionCode=KR)	회색	전 연령
음악	10	황색	10~20대

카테고리 라벨	YouTube categoryId	색상 배지	주 연령대
게임	20	파랑	10~20대
엔터	24	분홍	20~40대
뉴스	25	초록	40~60대
교육	27	보라	20~30대
뷰티	26 (Howto)	빨강	10~30대
먹방	26 (Howto/Food)	청록	20~40대
여행	19	하늘	30~50대
IT	28 (Science)	슬레이트	20~40대

4. 기술 스택

레이어	기술	이유
프론트엔드	HTML5 + CSS3 + Vanilla JS	프레임워크 없이 빠른 개발, 정적 파일 배포 가능
백엔드	FastAPI (Python 3.11+)	비동기 지원, YouTube API 연동 최적
YouTube API	google-api-python-client	공식 SDK, 쿼리 관리 내장
캐시	Redis (또는 파일 캐시)	1시간 캐시로 반복 API 호출 차단
DB (스냅샷)	SQLite	구독자 증가율 계산용 일별 스냅샷 저장
스케줄러	APScheduler	매일 09:00 자동 수집 + 스냅샷 저장
서버	Oracle Cloud Free Tier (Ubuntu 22.04)	영구 무료 서버
배포	Nginx + uvicorn	정적 파일 서빙 + FastAPI 프록시

STEP 1 — 정적 UI 구현 (샘플 HTML)

목표: 확정된 UI 샘플을 기반으로 HTML/CSS/JS 정적 페이지를 완성한다. 이 단계에서는 모든 데이터가 JS 내장 샘플이며 API 연동은 없다.

 **선행 조건: VS Code + Live Server 확장 설치 / 샘플 HTML 파일 참조**

#	작업 항목 및 설명	상태
1-1	프로젝트 폴더 구조 생성 frontend/ backend/ docs/ 폴더 + .gitignore + README.md	☐ 미완료
1-2	헤더바 구현 로고, 갱신 시각, 🔄 최신뉴스 갱신 버튼, 스피너 애니메이션	☐ 미완료
1-3	4개 탭 메뉴 구현 인기 영상 순위 / 구독자 순위 / 구독자 증가율 / 연령대 분석	☐ 미완료
1-4	인기 영상 탭 — 필터 구현 일간·주간·월간 / 10개 카테고리 필터 pill 버튼	☐ 미완료
1-5	1위 인라인 플레이어 구현 썸네일 + 재생버튼 → 클릭 시 YouTube embed iframe 교체	☐ 미완료
1-6	2~10위 리스트 구현 썸네일(실제 yting.com) + 제목 + 채널 + 조회수 + 새 탭 링크	☐ 미완료
1-7	구독자 순위 탭 구현 채널 아바타 + 이름 + 구독자 수 + 총 조회수 / 정렬 전환	☐ 미완료
1-8	구독자 증가율 탭 구현 일·주·월 전환 / 증가수·증가율 / SVG 스파크라인 추세	☐ 미완료
1-9	연령대 분석 탭 구현 시청 비중 바차트 / 선호 콘텐츠 / 추천 영상 5건	☐ 미완료

#	작업 항목 및 설명	상태
1-1 0	다크모드 대응 CSS prefers-color-scheme 적용	☐ 미완료

Claude Code 프롬프트 — STEP 1 — 정적 UI

지금까지 대화에서 확정된 KR 유튜브 트렌드 대시보드 UI를

완전한 standalone HTML 파일로 구현해줘.

파일: `frontend/index.html` (CSS·JS 모두 포함)

레이아웃:

- 헤더: 로고 (빨간 사각형+아이콘) + 갱신시각 + 갱신버튼
- 탭: 인기 영상 순위 / 구독자 순위 / 구독자 증가율 / 연령대 분석
- 인기 영상 탭:
 - 필터: 일간|주간|월간 + 전체|음악|게임|엔터|뉴스|교육|뷰티|먹방|여행|IT
 - 2단 그리드: 왼쪽=1위 인라인 플레이어, 오른쪽=2~10위 리스트
 - 1위 플레이 클릭 → YouTube embed iframe으로 교체 (autoplay)
 - 2~10위 클릭 → 새 탭으로 youtube.com 이동
 - 카테고리별 독립 데이터 10건씩 (전체·음악·게임·엔터·뉴스·교육·뷰티·먹방·여행·IT)
- 구독자 순위: 채널 아바타 + TOP 10 테이블

- 구독자 증가율: 일·주·월 전환 + SVG 스파크라인
- 연령대 분석: 2컬럼 (시청비중 + 선호콘텐츠) + 추천영상 5건

스타일: 흰 배경, 0.5px 테두리, 라운드 코너, 다크모드 지원

데이터: JS 내장 샘플 (실제 YouTube video ID 포함)

 완료 기준 (Gate Check)

☐	Live Server로 실행 시 4개 탭 모두 정상 렌더링
☐	카테고리 필터 클릭 → 해당 카테고리 독립 데이터 10건 표시
☐	1위 재생 버튼 클릭 → YouTube embed 인라인 재생 시작
☐	2~10위 클릭 → 새 탭으로 유튜브 정상 이동
☐	일·주·월 전환 → 구독자 증가율 데이터 교체
☐	연령대 필터 클릭 → 선호 콘텐츠 + 추천 영상 갱신
☐	다크모드에서 모든 텍스트 가독성 확인

STEP 2 — YouTube API 연동 (인기 영상)

목표: FastAPI 백엔드를 구축하고 YouTube Data API v3로 실제 한국 인기 영상을 수집해 프론트엔드에 연결한다.

 선행 조건: STEP 1 Gate Check 전부 통과 / Google Cloud 프로젝트 + API 키 발급

#	작업 항목 및 설명	상태
2-1	Google Cloud 프로젝트 생성 YouTube Data API v3 활성화 + API 키 발급 + .env 저장	☐ 미완료
2-2	FastAPI 앱 기본 구조 backend/main.py — CORS 설정 + /health 엔드포인트	☐ 미완료
2-3	YouTube API 래퍼 구현 backend/youtube_client.py — videos.list 호출 함수	☐ 미완료
2-4	GET /api/videos 구현 ?cat=all&period=daily 파라미터 → mostPopular + regionCode=KR	☐ 미완료
2-5	파일 캐시 구현 cache/{cat}_{period}.json 저장 / 1시간 유효기간 확인	☐ 미완료
2-6	프론트엔드 fetch 연결 JS 샘플 데이터 → fetch('/api/videos') 교체	☐ 미완료
2-7	쿼타 소비량 확인 Google Cloud Console → API 사용량 확인	☐ 미완료

 Claude Code 프롬프트 — STEP 2 — YouTube API 인기 영상

backend/main.py와 backend/youtube_client.py를 구현해줘.

```
[youtube_client.py]

from googleapiclient.discovery import build

import json, os, time

CATEGORY_MAP = {

    "all": "",    "music": "10", "game": "20",

    "ent": "24",  "news": "25",  "edu": "27",

    "beauty": "26", "food": "26", "travel": "19", "it": "28"

}

CACHE_DIR = "cache/"

CACHE_TTL = 3600  # 1시간

def get_popular_videos(cat="all", period="daily", max_results=10):

    cache_path = f"{CACHE_DIR}{cat}_{period}.json"

    # 캐시 유효 확인

    if os.path.exists(cache_path):

        if time.time() - os.path.getmtime(cache_path) < CACHE_TTL:

            return json.load(open(cache_path))
```

```

youtube = build("youtube", "v3", developerKey=os.getenv("YOUTUBE_API_KEY"))

params = dict(part="snippet,statistics", chart="mostPopular",

               regionCode="KR", maxResults=max_results)

if CATEGORY_MAP.get(cat):

    params["videoCategoryId"] = CATEGORY_MAP[cat]

res = youtube.videos().list(**params).execute()

items = [{"id":v["id"], "title":v["snippet"]["title"],

          "channel":v["snippet"]["channelTitle"],

          "views":v["statistics"].get("viewCount", "0"),

          "categoryId":v["snippet"]["categoryId"]}

          for v in res.get("items", [])]

os.makedirs(CACHE_DIR, exist_ok=True)

json.dump(items, open(cache_path, "w"), ensure_ascii=False)

return items

```

[main.py]

```

from fastapi import FastAPI

from fastapi.middleware.cors import CORSMiddleware

from youtube_client import get_popular_videos

```

```
app = FastAPI()

app.add_middleware(CORSMiddleware, allow_origins=["*"], ...)

@app.get("/api/videos")

def videos(cat:str="all", period:str="daily"):

    return get_popular_videos(cat, period)
```

실행: `uvicorn main:app --reload --port 8000`

✅ 완료 기준 (Gate Check)

- | | |
|--------------------------|--|
| ☐ | uvicorn 실행 후 /docs 접속 성공 |
| ☐ | GET /api/videos?cat=all&period=daily → 실제 한국 영상 10건 반환 |
| ☐ | GET /api/videos?cat=music → 음악 카테고리 영상 반환 |
| ☐ | 두 번째 호출 시 캐시 사용 (응답 속도 차이 확인) |
| ☐ | 프론트엔드에서 실제 영상 제목·썸네일 표시 |
| ☐ | Google Cloud Console 쿼타 소비량 30유닛 이하 확인 |

STEP 3 — 구독자 순위 & 증가율 연동

목표: 한국 주요 채널 구독자 수를 수집하고 SQLite에 일별 스냅샷을 저장해 증가율을 계산한다.

📌 선행 조건: STEP 2 Gate Check 전부 통과

#	작업 항목 및 설명	상태
3-1	추적 채널 ID 목록 파일 작성 config/channels.json — 한국 주요 채널 50개 ID 목록	☐ 미완료
3-2	GET /api/channels 구현 channels.list 50개 배치 1회 → 구독자·조회수 반환	☐ 미완료
3-3	SQLite DB 구조 생성 snapshots(id, channel_id, subscribers, views, created_at)	☐ 미완료
3-4	스냅샷 저장 함수 구현 매일 09:00 실행 → 전체 채널 현재 구독자 수 DB 저장	☐ 미완료
3-5	GET /api/growth 구현 ?period=daily weekly monthly → DB 과거값 - 현재값 계산	☐ 미완료
3-6	APScheduler 스케줄러 연결 startup 이벤트에 매일 09:00 수집 태스크 등록	☐ 미완료
3-7	프론트엔드 증가율 탭 연결 JS 샘플 → fetch('/api/growth?period=daily') 교체	☐ 미완료

💬 Claude Code 프롬프트 — STEP 3 — 구독자 & 증가율

backend/channel_tracker.py와 /api/channels, /api/growth를 구현해줘.

```

[channel_tracker.py]

import sqlite3, json

from googleapiclient.discovery import build

CHANNELS_FILE = "config/channels.json" # [{id, name}] 목록

DB_PATH = "data/snapshots.db"

def init_db():

    conn = sqlite3.connect(DB_PATH)

    conn.execute("""CREATE TABLE IF NOT EXISTS snapshots

        (id INTEGER PRIMARY KEY, channel_id TEXT,

         subscribers INTEGER, views INTEGER, created_at DATE)""")

    conn.commit(); conn.close()

def fetch_and_save_snapshots():

    channels = json.load(open(CHANNELS_FILE))

    ids = [c["id"] for c in channels]

    # 50개 배치 1회 호출 = 1유닛

    youtube = build("youtube", "v3", developerKey=...)

    res = youtube.channels().list(

```



```

        part="statistics", id=",".join(ids[:50])

    ).execute()

conn = sqlite3.connect(DB_PATH)

from datetime import date

today = str(date.today())

for item in res["items"]:

    conn.execute("INSERT INTO snapshots VALUES (NULL,?,?,?,?)",

        (item["id"],

            int(item["statistics"]["subscriberCount"]),

            int(item["statistics"]["viewCount"]), today))

conn.commit(); conn.close()

def get_growth(period="daily"):

    days = {"daily":1,"weekly":7,"monthly":30}[period]

    conn = sqlite3.connect(DB_PATH)

    # 오늘 vs N일 전 비교 쿼리

    rows = conn.execute("""

        SELECT a.channel_id,

            a.subscribers - b.subscribers as gain,

            ROUND((a.subscribers - b.subscribers)*100.0/b.subscribers, 2) as
rate

        FROM snapshots a JOIN snapshots b

```

```

        ON a.channel_id=b.channel_id

        WHERE a.created_at=date('now')

        AND b.created_at=date('now',?||' days')

        ORDER BY gain DESC LIMIT 10

        """ , (f"-{days}",)).fetchall()

    conn.close()

    return [{"channel_id":r[0],"gain":r[1],"rate":r[2]} for r in rows]

```

[main.py 추가]

```

@app.get("/api/channels")

def channels(): return fetch_channels_list()

@app.get("/api/growth")

def growth(period:str="daily"): return get_growth(period)

```

✅ 완료 기준 (Gate Check)

- | | |
|--------------------------|---|
| ☐ | GET /api/channels → 구독자 기준 TOP 10 채널 반환 |
| ☐ | DB에 오늘 날짜 스냅샷 저장 확인 |
| ☐ | 2일 이상 데이터 쌓인 후 GET /api/growth → 증가량 반환 |
| ☐ | 프론트엔드 구독자 순위 탭 실제 데이터 표시 |
| ☐ | 프론트엔드 증가율 탭 일·주·월 전환 정상 동작 |

STEP 4 — 서버 배포 & 자동화

목표: Oracle Cloud Free 서버에 배포하고 매일 자동 갱신 스케줄러를 운영한다.

📌 선행 조건: STEP 3 Gate Check 전부 통과 / Oracle Cloud 계정

#	작업 항목 및 설명	상태
4-1	Oracle Cloud Free Tier VM 생성 Ubuntu 22.04, VM.Standard.E2.1.Micro, Always Free	☐ 미완료
4-2	서버 환경 구성 Python 3.11, Nginx, pip install -r requirements.txt	☐ 미완료
4-3	코드 배포 및 .env 설정 git clone + .env에 YOUTUBE_API_KEY 입력	☐ 미완료
4-4	systemd 서비스 등록 yttrends.service — 서버 재시작 시 자동 실행	☐ 미완료
4-5	Nginx 리버스 프록시 설정 / → index.html / /api → uvicorn:8000	☐ 미완료
4-6	초기 DB 스냅샷 수동 실행 python -c 'from channel_tracker import *; fetch_and_save_snapshots()'	☐ 미완료
4-7	E2E 최종 검증 실제 서버 URL에서 전체 기능 동작 확인	☐ 미완료

💬 Claude Code 프롬프트 — STEP 4 — 서버 배포

배포를 위한 파일 3개를 만들어줘.

1. deploy.sh

```
- apt update && apt install -y python3.11 nginx git  
  
- git clone {repo} && cd yttrends  
  
- pip install -r requirements.txt  
  
- .env 파일 생성 안내 출력
```

2. yttrends.service (systemd)

```
[Unit] Description=YT Trends Dashboard  
  
[Service]  
  
WorkingDirectory=/home/ubuntu/ytrends/backend  
  
ExecStart=/usr/bin/uvicorn main:app --host 0.0.0.0 --port 8000  
  
Restart=always
```

3. nginx.conf

```
server {  
  
    listen 80;  
  
    location / { root /home/ubuntu/ytrends/frontend; }  
  
    location /api { proxy_pass http://127.0.0.1:8000; }  
  
}
```

4. requirements.txt 최종 버전

```
fastapi, uvicorn, google-api-python-client,  
  
google-auth, apscheduler, python-dotenv
```

✅ 완료 기준 (Gate Check)

☐	서버 IP/도메인 접속 시 대시보드 정상 렌더링
☐	실제 한국 인기 영상 TOP 10 표시
☐	서버 reboot 후 자동 실행 확인
☐	매일 09:00 스냅샷 저장 로그 확인
☐	Google Cloud Console 일 쿼타 소비량 200유닛 이하 확인

STEP 5 — 고도화 & 안정화

목표: UX 개선 및 에러 처리를 강화하고 모니터링 체계를 구축한다.

📌 선행 조건: STEP 4 Gate Check 전부 통과 / 최소 3일 운영 후 진행

#	작업 항목 및 설명	상태
5-1	로딩 스켈레톤 UI API 응답 대기 중 썸네일 플레이스홀더 표시	☐ 미완료
5-2	에러 처리 강화 API 실패 시 캐시 데이터 폴백 + 사용자 안내 메시지	☐ 미완료
5-3	쿼타 모니터링 알림 일 8,000유닛 초과 시 이메일/슬랙 알림	☐ 미완료
5-4	채널 목록 관리 UI 관리자용 채널 추가/삭제 간단 UI (선택사항)	☐ 미완료
5-5	OG 태그 & SEO <meta og:title/description/image> 설정	☐ 미완료
5-6	모바일 반응형 최적화 breakpoint 600px 이하 레이아웃 조정	☐ 미완료
5-7	성능 측정 Lighthouse 점수 90+ 목표	☐ 미완료

6. 전체 진행 현황 대시보드

Phase	STEP	내용	기간	Gate	완료일
Phase 1	STEP 1	정적 UI 구현 (샘플 HTML)	1주차	☐	
Phase 1	STEP 2	YouTube API — 인기 영상 연동	2주차	☐	
	—	실제 인기 영상 데이터 표시	2주차 말		
Phase 1 완료					
Phase 2	STEP 3	구독자 순위 & 증가율 연동	3주차	☐	
Phase 2	STEP 4	서버 배포 & 자동화	4주차	☐	
Phase 2	STEP 5	고도화 & 안정화	5주차	☐	
	— 전체 완료 —	운영 서비스 오픈	5주차 말		

💡 **STEP 1~2만 완료해도 실제 YouTube API 데이터로 동작하는 인기 영상 대시보드가 완성됩니다. STEP 3~5는 증가율 분석과 운영 안정화 단계입니다.**

항목	목표값	비고
일 API 쿼타 소비	200유닛 이하	10,000유닛의 2%
페이지 로드 속도	2초 이내	캐시 적중 시 0.5초
1위 영상 재생	클릭 후 1초 내 시작	YouTube embed autoplay
월 서버 운영 비용	\$0	Oracle Cloud Free Tier
데이터 갱신 주기	1시간 (인기 영상) / 1일 (구독자)	APScheduler 자동화

⚠️ **저작권 및 이용약관: YouTube API는 Google API Services 이용약관을 따릅니다. 수집한 데이터를 재배포하**

거나 상업적으로 판매하는 것은 금지됩니다.

© 2026 KR 유튜브 트렌드 대시보드 *PRD v1.0* — 바이브 코딩 에디션